Quran under 1 MB
Implementation, Response Measurements and Optimization

quran-under1mb contributors

November 2026

Abstract

This offline Android reader contains Arabic, English and Hindi text in an 881,355-byte APK. It uses native Java views and XZ compression. Compatibility was checked with 195 framework tests across Android API 24–36. Host-side response measurements are reported separately from physical-phone performance, which has not been measured. Code, data checks, build tooling and benchmark source are available in the accompanying repository.

1 Response time: evidence first

No physical-device cold-start, human tap-to-display latency, frame-rate, battery or RAM measurements have been collected. A zero-lag claim would therefore be unsupported. The timings below describe a Linux cloud Robolectric harness using Android API 36 and native graphics. Each workload has five warm-ups and thirty recorded samples. Median and nearest-rank 95th-percentile values are reported. The JVM and host file cache are warm.

Host harness operation	Median ms	p95 ms
Activity creation and layout	132.855	186.431
New-group reader opening	126.567	174.341
Cached reader opening	32.671	34.779
Progress commit and UI callback	35.413	48.831

Fresh Activity creation includes setup and an explicit layout; it is not Android process startup. New-group reader opening clears the chapter cache and decoded group before opening Al-Faatiha, awaiting the worker and laying out the reader. Cached opening retains those caches. The save workload changes progress, waits for a storage commit and the UI callback, and lays out the reader. Harness overhead is included. Raw samples and environment details are in `docs/benchmarks.json`.

2 What is shipped

Property	Value
Signed APK bytes	881,355
Decimal MB / KiB	0.881355 / 860.7
Margin below 1,000,000 bytes	118,645
Surahs / verses per language	114 / 6,236
Languages	Arabic, English, Hindi
Compressed text bytes	775,704
Text share of APK	88.01%
Decoded grouped UTF-8 bytes	4,726,844
Minimum / target API	24 / 36

APK size is distinct from installed storage and peak memory. No audio, embedded fonts, AI model, advertising SDK or analytics is bundled. The app requests no Android permissions. Reading needs neither an account nor a server.

3 Implementation and technology

The app uses Java, one Activity, native ListView/TextView widgets and system fonts. XZ for Java 1.10 is the only external runtime library. OpenJDK 21, Python 3, Android platform/build tools 36, Maven, JUnit and Robolectric are build/test tools and are not shipped as app runtimes.

3.1 Loading and data format

Startup reads a 14,705-byte index rather than parsing the corpus. One background worker loads requested chapter data. Three XZ groups contain surahs 1–40, 41–80 and 81–114. A big-endian 32-bit length precedes each chapter's UTF-8 JSON array. Verse keys are `a`, `e` and `h`. Verse strings are preserved unchanged. The current group is retained as strings; a separate LRU cache holds at most three parsed chapters. Rows are recycled during scrolling.

3.2 Progress and user interaction

The footer's visible range comes from actual text geometry, not an estimated row boundary. Read progress is explicit user confirmation; visibility alone does not establish that a person read something. The single Mark read control presents verse numbers for confirmation. Finish surah explicitly handles all remaining verses without requiring a following verse or an estimated end-of-scroll state.

Per-verse read flags are serialized to private SharedPreferences. They are not just a highest verse number, so an unread gap need not count as read. The worker commits progress before the UI reports Saved. Pending writes are allowed to finish after Activity destruction. Bookmarks and reading position are independent of completion.

4 Compatibility and validation

Fifteen scenarios passed on each of API 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35 and 36: 195 framework checks. These represent Android 7.0, 7.1, 8.0, 8.1, 9, 10, 11, 12, 12L, 13, 14, 15 and 16. Android 7 uses legacy graphics in the harness; Android 8–16 use native graphics. This is not physical-device or full-OS-emulator testing.

Tests cover launch, language/font settings, bookmarks, position, invalid input, cancellation, duplicate saves, final verses, selection snapshots and persistence. The Al-Faatiha case starts at 5/7, confirms verses 6 and 7, checks 7/7, verifies persisted XML and reopens the Activity. Another case closes the Activity while a save is pending. API-specific system-bar handling and version-qualified startup resources support older releases. The release uses DEX 037 and verified APK v2/v3 signing.

5 Size analysis and further optimization

Compressed text dominates package size. Stored DEX occupies 89,245 bytes. ZIP entry sums exclude signing blocks and alignment/ZIP overhead, and do not equal the final APK size.

Chapters/group	Groups	XZ bytes	Max decoded bytes
1	114	1,176,340	362,251
10	12	894,396	1,696,818
20	6	831,248	2,527,302
40	3	775,704	3,757,975
114	1	729,308	4,726,844

Forty chapters per group is a compromise between compression and decode scope. One whole-corpus group saves 46,396 payload bytes, roughly 5.26% of the current APK with other costs held fixed. The largest decoded group would grow from 3,757,975 to 4,726,844 bytes. This experiment does not measure revised startup, heap use, code size or signing/alignment costs; it has not been adopted in the release.

Further candidates are shrinker-based removal of unused encoder classes, a tested compression-library upgrade, compact corpus encoding and dictionary-size experiments. Their savings are unmeasured here. Each requires corpus verification and regressions. Removing translations, diacritics or verses is not an acceptable optimization. Real-device frame and memory profiling should precede claims of smoother scrolling.

6 Install, build and reproduce

Download `downloads/Quran.apk`, verify `SHA256SUMS`, and open the file on Android 7 or later. Allow the browser/file manager to install apps if Android asks. Updates signed by the same maintainer key preserve app data. Uninstalling or clearing data removes local bookmarks/progress. The private release signing key is not in this repository. Source builds use a development key unless another key is supplied.

```
# OpenJDK 21; Python 3; SDK platform/build-tools 36
export ANDROID_HOME=/path/to/android-sdk
python3 scripts/verify_data.py
python3 scripts/build_android.py
python3 scripts/run_tests.py --matrix
python3 scripts/run_tests.py --benchmark
python3 scripts/analyze_size.py
# Copy the new benchmark JSON into docs/ before rendering.
python3 scripts/render_docs.py
cd docs
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The build script verifies the pinned dependency hash and enforces the sub-1,000,000-byte cap. Different signing keys and auxiliary notices can change a source build's bytes; bit-identical reproduction of the signed release is not claimed.

7 Openness, attribution and limits

Application code, scripts and documentation use MIT. Text and translations retain upstream terms, not MIT. Arabic text is attributed to Tanzil and preserved verbatim. English is Saheeh International; Hindi is by Suhel Farooq Khan and Saifur Rahman Nadwi. Data came through AlQuran Cloud. Translation terms include non-commercial-use conditions. See the third-party notices before reusing the corpus.

Preferences stay in private app storage and are not encrypted by the app. There is no account or remote backup. Installed storage, battery use, physical-device RAM, frame times and human input latency remain unmeasured. The repository provides a real-device protocol to collect that evidence.

References

1. https://github.com/umairrrkhan/quran-under1mb
2. https://tanzil.net/docs/text_license
3. https://tanzil.net/trans/
4. https://alquran.cloud/terms-and-conditions
5. https://tukaani.org/xz/java.html
6. https://robolectric.org/